OmniDN-S: A Small Network for Blind Image Restoration

m25csa016, IIT Jodhpur — Mini Competition: A Small Generative Model for Image Denoising

1. Introduction

The task is to restore a corrupted 256×256 RGB image whose degradation type and severity are unknown. Images come from natural photography, low-light imagery and medical imaging. Submissions are ranked independently on FLOPs, PSNR, SSIM and DISTS. Because FLOPs counts as much as each quality metric, we aimed for a model of about 3 GFLOPs that still generalises across content and degradation types. Our approach rests on three ideas: a compact U-Net that spends most of its computation at low resolution, a broad on-the-fly degradation pipeline, and fine-tuning with losses that match the evaluation metrics.

2. Architecture

OmniDN-S is a U-Net built from NAFNet blocks [1]. Each block applies LayerNorm, a 1×1 convolution, a 3×3 depthwise convolution, SimpleGate, simplified channel attention and a 1×1 convolution, followed by a gated feed-forward layer.

Encoder and decoder. The encoder has four stages with 16, 32, 64 and 128 channels and 1, 1, 2 and 4 blocks, each ending with a stride-2 convolution, so the bottleneck works at 16×16 with 256 channels. The decoder upsamples with a 1×1 convolution and PixelShuffle, adds the skip connection, and uses one block per level.

Computation at low resolution. Channels double whenever the resolution halves, so a block costs about the same at every level, but at 16×16 its receptive field covers far more of the image. We therefore place 12 of the 24 blocks in the bottleneck, which is important for motion and defocus blur.

Additions to NAFNet. Two of the bottleneck blocks are global multi-head self-attention over all 256 tokens, giving every output pixel an image-wide context. Two coordinate channels are concatenated to the input (CoordConv [6]), because chromatic aberration and lens blur depend on the distance from the image centre. A global residual connection adds the input to the output.

Size and output. The network has 6.66 M parameters and costs 2.963 GFLOPs at 1×3×256×256 (fvcore; ptflops reports 3.01 GMac). The output is clamped to [0, 1] and rounded to the 256 uint8 levels inside the network, so any float-to-uint8 conversion gives the same image. All heavy operations are standard convolutions and matrix multiplications that the FLOP counters see. There is no input-dependent computation and no test-time augmentation.

3. Training data

The training set has 75,602 samples from 21 public datasets: 60,948 images plus 14,654 native-resolution crops of large images. Images are stored as PNG with the long side at most 512 px.

Domain	Samples	Sources
Natural	31,884	LSDIR, Flickr2K, DIV2K, FFHQ, normal-light LOL-v2 / SICE
Medical	40,953	Chest X-ray (NIH-14, paediatric, Shenzhen, Montgomery), dental X-ray, MRI (IXI T1/T2, brain), CT (LoDoPaB), ultrasound (BUSI, HC18), histology, fundus, dermoscopy
Low-light	2,765	Dark LOL-v2 and SICE frames

Sampling. Batches draw natural, medical and low-light images with probability 0.55, 0.33 and 0.12. In addition, 10% of natural images are darkened before corruption, with the target darkened too, so the model learns to keep dark images dark.

Patches. Training patches of 256×256 are random crops after random downscaling, whole images resized to 256×256, or native-resolution crops.

Held-out images. 1,108 images are never used for training: Kodak24, CBSD68, McMaster, Urban100, DIV2K-val, LSDIR-val, LOL-v2 test and held-out medical images. From them we built two evaluation sets using plain skimage / OpenCV / PIL calls, not our training code: a proxy test set of 600 pairs with the listed degradations and their mixtures, and a set of 140 pairs with corruption types never used in training (glass blur, zoom blur, uniform noise, JPEG2000 and rare variants).

4. Degradations used during training

Every training pair is generated on the fly. A chain of 0 to 4 operators is drawn: 0 with probability 0.04 (clean input, so the model learns to leave clean images unchanged), 1 with 0.42, 2 with 0.34, 3 with 0.16 and 4 with 0.04.

Ordering. In 80% of chains the operators follow physical order: optics, then resampling, then sensor noise, then artefacts, then compression. In 20% they are shuffled, as in BSRGAN [5].

Spatial mixing. In 8% of samples a second chain is applied to a random region of the image.

Quantisation. The result is rounded to uint8, as the test images will be.

Library defaults. For about 30% of draws an operator uses the default parameters of the common library call, for example `skimage.util.random_noise` defaults (Gaussian σ = 25.5/255, salt-and-pepper amount 0.05, speckle variance 0.01) or `cv2.GaussianBlur` with kernel sizes 3–15.

The 21 operators are:

Family	Operators and severity ranges
Noise	Gaussian (σ 1–75/255; per channel, gray or channel-correlated), spatially correlated noise, Poisson (skimage rule or peak 2–500), Poisson-Gaussian, camera noise (Bayer mosaic + demosaicing), row/column banding, speckle (var 0.003–0.3), salt-and-pepper / impulse (amount 0.003–0.25), Rician (MRI), chroma noise
Blur	Gaussian (isotropic and anisotropic, σ 0.3–4), motion (line kernels 3–31 px and random camera-shake paths), defocus (disk radius 1–8, box, median)
Optics and resampling	chromatic aberration (radial R/B scaling, channel shift, lens distortion, axial blur), down–up resampling (×0.25–0.9), pixelation
Compression	JPEG (quality 5–95, optional double compression), WebP

Family	Operators and severity ranges
Artefacts	ringing / over-sharpening, posterisation and dithering, dust, scratches and dead pixels

Excluded. Brightness, contrast and gamma changes, vignetting and haze were left out on purpose: a blind model cannot tell them apart from naturally dark or flat content, and "correcting" them would damage clean low-light and medical images.

Stage-2 mix. In the fine-tuning stages the sampling weights of the task's named types (Gaussian noise, JPEG, blur) were increased.

5. Training objective and schedule

Training ran in three stages on single A30 GPUs, with batch 32, 256×256 patches, AdamW, an exponential moving average of the weights (decay 0.999) and bf16 mixed precision.

Stage 1: fidelity training. Charbonnier loss plus 0.05 × L1 distance between Fourier spectra; learning rate 1e-3, cosine schedule over 20 hours (669,511 iterations).

Stage 2a: metric-aligned fine-tuning. The loss matches the evaluation metrics:

```
L = (1/B) Σ_i 10·log10(MSE_i) + 30·(1 − SSIM)
```

The first term is exactly minus the mean per-image PSNR. The SSIM term re-implements skimage's default SSIM (7×7 uniform window, sample covariance) and agrees with it to about 1e-5. Learning rate 1e-4 for 1.5 hours.

Stage 2b: perceptual fine-tuning. Two runs start from stage 2a and add $\lambda \cdot$DISTS [3], with $\lambda = 15$ and $\lambda = 40$. The loss uses the official DISTS network, frozen and in fp32.

Weight blending. The final weights are $0.55 \cdot \theta(\lambda = 15) + 0.45 \cdot \theta(\lambda = 40)$. Blending network weights [4] lets us choose the trade-off between fidelity (PSNR, SSIM) and perceptual quality (DISTS) at no extra inference cost. This perceptual stage is our reading of the "generative" part of the task: the network stays a single deterministic forward pass, since diffusion or GAN models would exceed the FLOP budget or cost PSNR and SSIM.

6. Results

All numbers below use the organisers' metric implementations (skimage PSNR/SSIM on uint8 images, official DISTS). They are measured on our own held-out sets, not on the hidden test set.

Model	GFLOPs	Proxy test (600): PSNR / SSIM / DISTS	Unseen corruptions (140)
Input, no restoration	0	25.43 / 0.618 / 0.283	25.59 / 0.592 / 0.289
SCUNet, public pretrained [2]	79.0	30.03 / 0.807 / 0.230	30.25 / 0.806 / 0.230
OmniDN-S, stage 1	2.96	33.96 / 0.904 / 0.150	31.54 / 0.860 / 0.192
OmniDN-S, stage 2a	2.96	34.04 / 0.906 / 0.147	31.51 / 0.860 / 0.191
OmniDN-S, submitted	**2.96**	**33.92 / 0.903 / 0.117**	**31.38 / 0.857 / 0.166**
OmniDN-M (width 32, not submitted)	11.36	34.42 / 0.910 / 0.107	31.60 / 0.860 / 0.162

By domain. The submitted model scores 30.25 dB on natural, 38.34 dB on low-light and 36.59 dB on medical images.

Against SCUNet. OmniDN-S matches SCUNet within 0.4 dB on Gaussian noise and JPEG, and is clearly better on blur, salt-and-pepper noise and chromatic aberration, which SCUNet was not trained for.

Unseen corruptions. Uniform noise and unseen mixtures are restored well. Glass blur, zoom blur and JPEG2000 artefacts are not improved.

Extreme severities. At severities beyond the training range (for example $\sigma = 75$ noise, JPEG quality 5, 25% salt-and-pepper) the model still improves PSNR by 2–22 dB.

Ablations and design decisions (short fine-tunes of OmniDN-S, proxy test set):

- *DISTS weight.* $\lambda = 40$ lowered DISTS from 0.157 to 0.124 for –0.27 dB PSNR. $\lambda = 120$ gave only 0.007 more DISTS for a further –0.29 dB.
- *Weight blending.* Blending the fidelity and perceptual weights recovered most of the PSNR while keeping most of the DISTS gain; for example, a 50% blend kept 43% of the DISTS gain for 24% of the PSNR drop.
- *Distillation.* Distilling from the larger OmniDN-M was 2.4× slower and gave –0.05 dB at equal training time, so it was not used.
- *SSIM weight.* Raising the SSIM weight from 30 to 100 gained only 0.0009 SSIM for –0.06 dB, so 30 was kept.
- *Model size.* OmniDN-M is 0.50 dB better on the proxy set but only 0.22 dB better on unseen corruptions, at 3.8× the FLOPs. A Monte-Carlo simulation of the four-metric ranking against plausible competitor fields favoured OmniDN-S in most scenarios, so we submitted the small model.

7. Submission

The package contains `model.py` (network), `weights/omnidn.pth` (PyTorch state_dict), `inference.py` (256×256×3 image in, 256×256×3 uint8 image out) and `check_flops.py` (fvcore and ptflops). It was verified end to end with the organisers' metric calls on GPU and CPU and reproduces the numbers above.

References

[1] L. Chen et al., Simple Baselines for Image Restoration (NAFNet), ECCV 2022.
[2] K. Zhang et al., Practical Blind Image Denoising via Swin-Conv-UNet and Data Synthesis (SCUNet), MIR 2023.
[3] K. Ding et al., Image Quality Assessment: Unifying Structure and Texture Similarity (DISTS), TPAMI 2022.
[4] X. Wang et al., ESRGAN: Enhanced Super-Resolution GANs (network interpolation), ECCV Workshops 2018.
[5] K. Zhang et al., Designing a Practical Degradation Model for Deep Blind Image Super-Resolution (BSRGAN), ICCV 2021.
[6] R. Liu et al., An Intriguing Failing of Convolutional Neural Networks and the CoordConv Solution, NeurIPS 2018.